

LATENCY-AWARE AGENTS

From Deployment Profiling to Profile-Conditioned Control

Proposed paper narrative, section draft, and figure placement

Core thesis

Real deployment latency should be treated as a first-class condition of an agent, not as an implementation nuisance. We standardize device-specific temporal latency profiles, use controlled replay to predict how candidate policies will perform on new hardware, and then train profile-conditioned agents that adapt their actions to the deployment clock. Memory and representation analysis explain what information these agents still need to act robustly.

Three-part narrative

- 1. Standardize and predict:** Convert a new device into a reproducible temporal profile and predict policy performance through replay.
- 2. Train latency-aware agents:** Use the predicted deployment condition as a training and checkpoint-selection distribution.
- 3. Explain what awareness requires:** Study explicit conditioning, temporal memory, representations, and transfer.

Source note. Empirical figures are reproduced from the supplied experiment HTML artifacts. The profiling panels in Xinyuan's Google Slides were not available as an exported image file, so Sections 1–2 use faithful draft schematics and reserve the empirical panel positions for the original slide figures.

1. Latency-Aware Agents: Problem and Framework

A latency-aware agent is a policy that treats the deployment clock as part of its operating condition. Its observations may be stale, its actions may be executed later than intended, and the delay process can change with the model, GPU, runtime, batching policy, and system load. A policy that is strong in a synchronous simulator may therefore fail after deployment even when its perception and action spaces are unchanged.

The central difficulty is data collection. Training directly on every physical deployment configuration requires long real-time rollouts and repeats the entire data-generation cycle whenever the GPU or systems stack changes. Our proposed framework separates a small measurement stage from a large controlled training stage: collect the latency process on the new hardware, standardize it, replay it to predict candidate-policy performance, and use the resulting profile as the distribution for training and model selection.

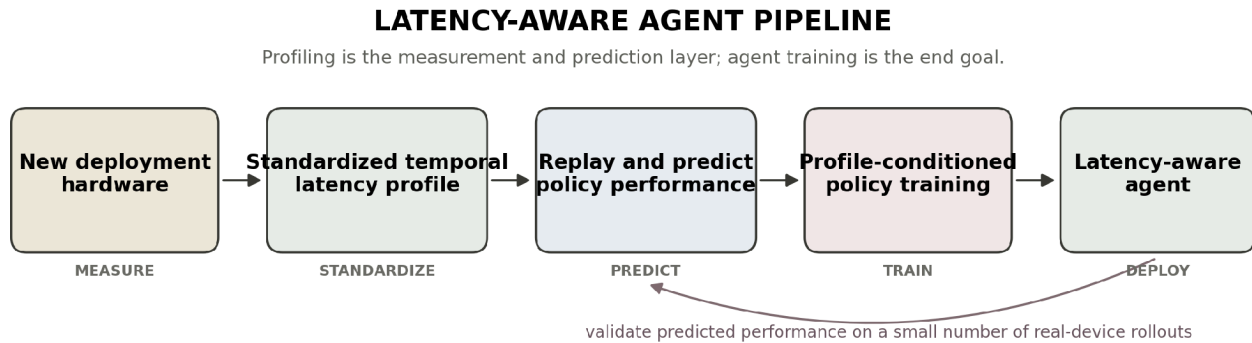


Figure 1. From deployment profiling to latency-aware agents. A device-specific temporal profile is standardized and replayed to predict policy performance; the same profile then becomes the training and checkpoint-selection condition for a latency-aware policy. Draft overview schematic.

Main framing sentence. Profiling is not the endpoint of the paper. It is the measurement and prediction layer that makes latency-aware agent training feasible.

2. Standardizing Deployment Latency and Predicting Policy Performance

This section is the protocol core of the paper. The goal is to convert a deployment-specific systems phenomenon into a standardized object that can be compared, replayed, and used to evaluate policies. Xinyuan’s experiments motivate three requirements: the same model can exhibit different latency profiles across devices; latency has temporal structure that is lost under IID or fixed-delay approximations; and replay should reproduce the behavioral effect of the source profile on a new target system.

2.1 Device-specific temporal profiling

For each model–device–runtime configuration, the protocol records step-wise inference latency together with the decision index and relevant runtime metadata. The resulting artifact is a temporal trace rather than a single mean. The primary empirical panel should compare traces and marginal distributions across devices, followed by a temporal statistic such as autocorrelation or burst structure. The claim is that deployment profiles differ not only in scale but also in temporal organization.

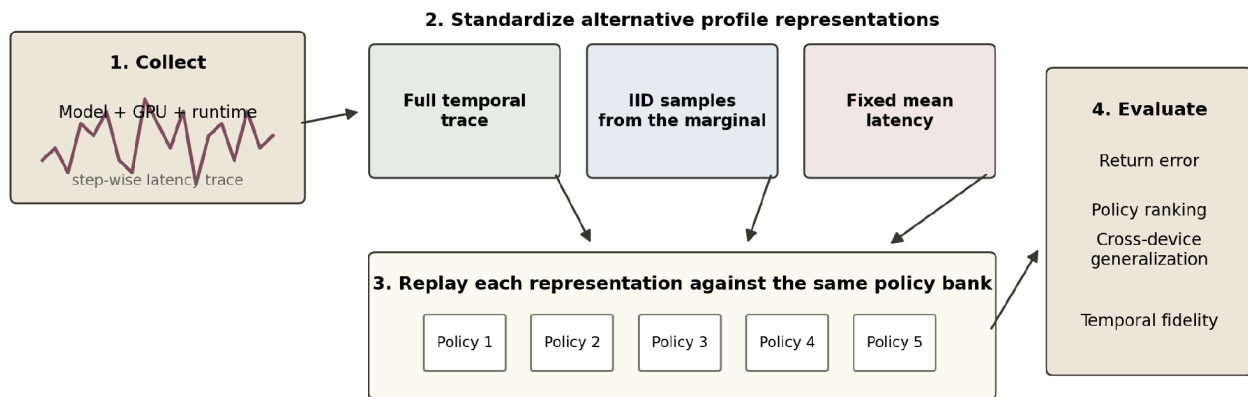
2.2 Replay-based performance prediction

Given a profile collected from a new GPU, we replay the profile in a controlled environment and evaluate a bank of candidate policies. The resulting returns are treated as predictions of real-device performance. The strongest validation is policy-level: replay should predict both absolute return and which policy will perform best. This matters because the practical decision is often model or checkpoint selection, not only estimation of one scalar return.

2.3 Evaluating a profiling method through policy behavior

A profile representation should be judged by whether it preserves policy behavior. We therefore compare the full temporal trace with simplified alternatives, including IID samples from the same marginal distribution and a fixed delay equal to the same mean. Each representation is replayed against the same policy bank, and we compare return error, policy ranking, and transfer to new devices. This provides a principled way to evaluate both policies and profiling methods in one protocol.

STANDARDIZING AND EVALUATING A DEPLOYMENT PROFILE



A profile representation is good when it preserves policy behavior, not merely its mean latency.

Figure 2. Standardizing and evaluating a deployment latency profile. Full temporal, IID, and fixed-mean representations are replayed against the same policy bank. A good representation preserves deployment returns and policy rankings, not only latency summary statistics. Draft schematic based on Xinyuan’s profiling and sim-to-real calibration experiments.

Insert original Xinyuan panels here. Recommended empirical layout: (a) latency traces across devices, (b) temporal versus IID profile comparison, (c) real-device return versus replay-predicted return, and (d) policy-ranking agreement. No numerical results are fabricated in this draft.

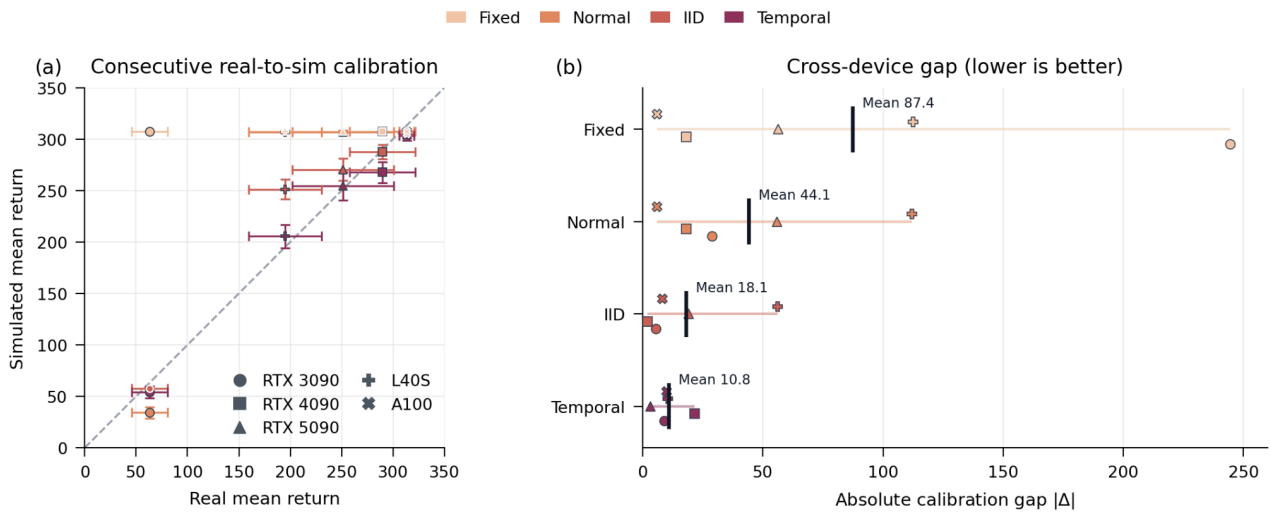
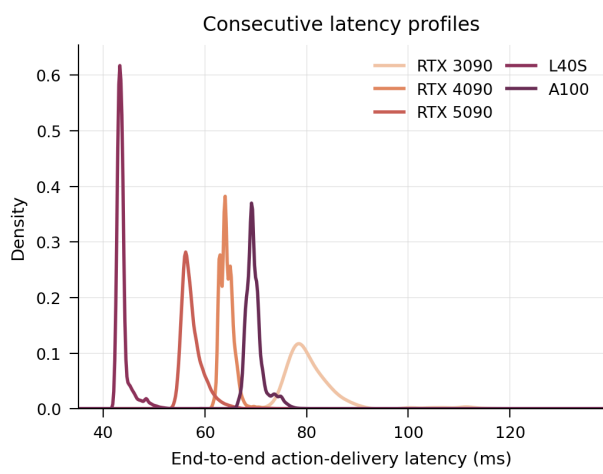


Table 1 at the end of Section 2. Summarize each deployment profile with device/GPU, runtime, mean and tail latency, one temporal statistic, real-device return, replay-predicted return, and policy-ranking agreement.



3. Training Latency-Aware Agents from Predicted Deployment Profiles

Once a target profile can be reproduced and its policy effect can be predicted, the protocol becomes a training interface. The target deployment profile is injected during optimization and checkpoint selection, replacing repeated real-device data collection with controlled profile-conditioned rollouts. This section first establishes that current VLAs are not latency-aware, then studies whether latency-diverse and explicitly conditioned training can close the gap.

3.1 Existing agents are brittle to latency shift

Across OpenVLA, $\pi 0.5$, and GR00T, policies trained in low- or zero-latency conditions lose performance as evaluation latency increases. The result is visible across Deadly Corridor, Demon Attack, and Flappy Bird, demonstrating a cross-model deployment problem rather than a checkpoint-specific failure.

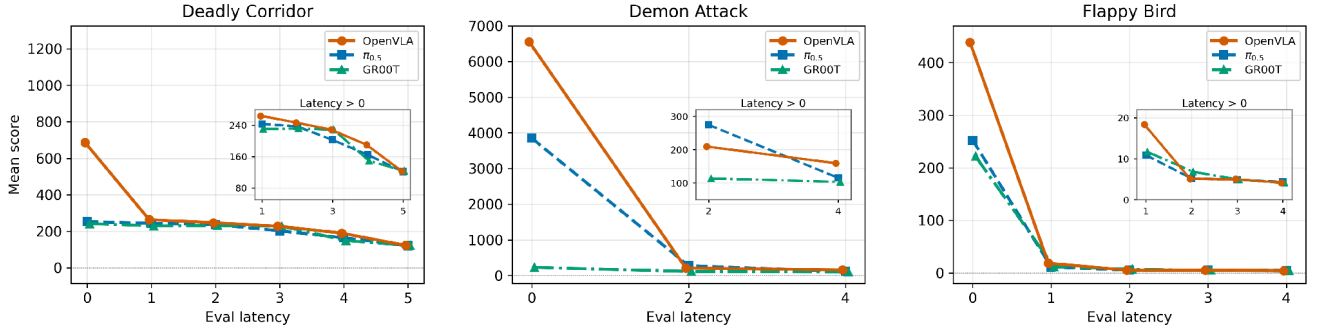
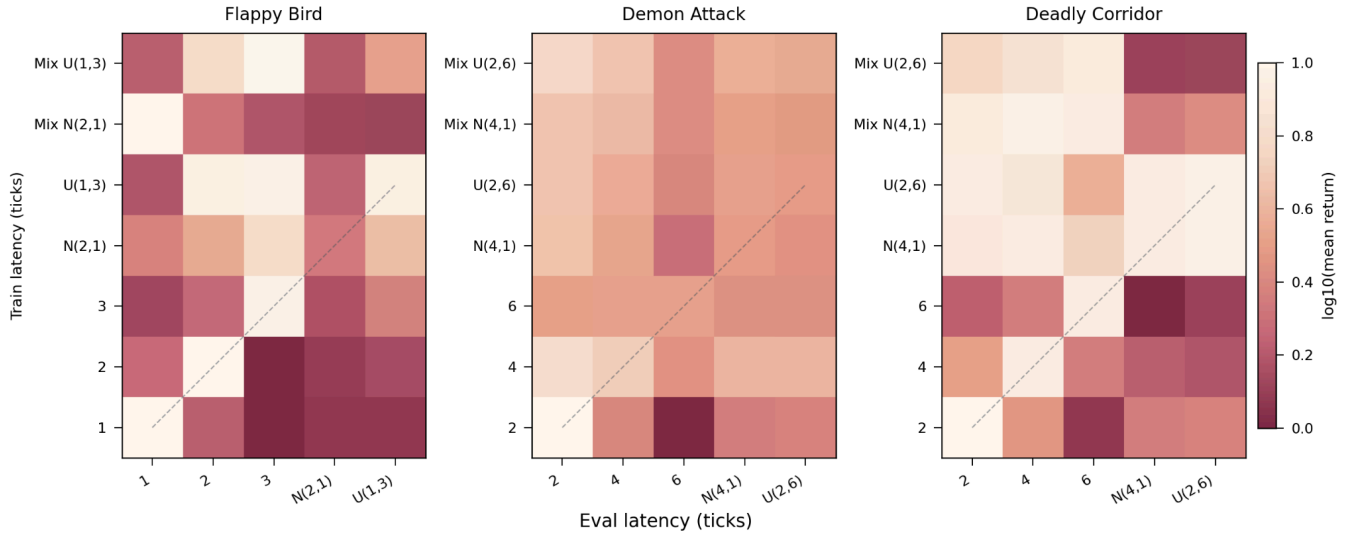


Figure 3. Cross-model degradation under increasing evaluation latency. Policies trained without deployment delay are brittle across three VLA families and three tasks. Source: standalone.html.

These results establish that existing VLAs are brittle to deployment latency. However, systematically crossing different training and evaluation profiles with VLAs is computationally expensive. We therefore first study latency-aware training in smaller agents, where fixed and stochastic latency profiles can be evaluated under controlled conditions.

3.2 Small-model study: broader latency training improves robustness, but not uniformly

We first use small agents to systematically evaluate generalization across fixed, stochastic, and mixed latency profiles. Each policy is trained under one latency condition and evaluated across multiple fixed and stochastic profiles.



First, latency robustness often generalizes asymmetrically from harder to easier conditions. Policies trained at higher fixed latency frequently retain performance when evaluated at shorter delays, whereas policies trained at low latency are more brittle under longer delays.

Second, stochastic and mixed-profile training can improve coverage across multiple evaluation conditions. In particular, policies trained with stochastic latency often generalize to fixed latency values covered by the training distribution. However, the benefit is task-dependent, and no single training profile dominates uniformly across all tasks.

3.3 Scaling latency-aware training to VLAs

We first train OpenVLA policies under different fixed latencies and evaluate each checkpoint across the same latency grid. As in the small-model study, policies generally perform best at or near the latency observed during training. Higher-latency specialists sometimes transfer to shorter delays, while low-latency specialists degrade sharply under longer delays.

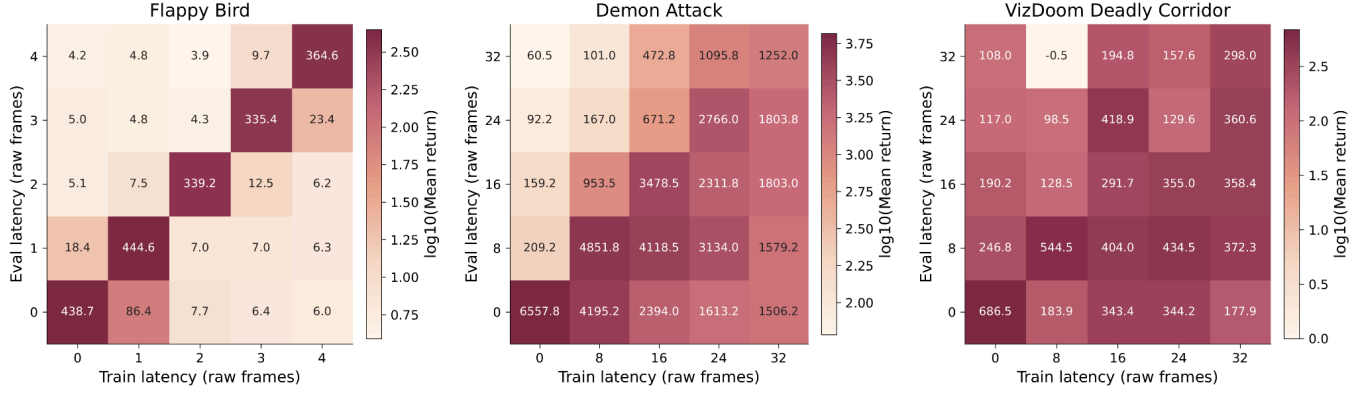
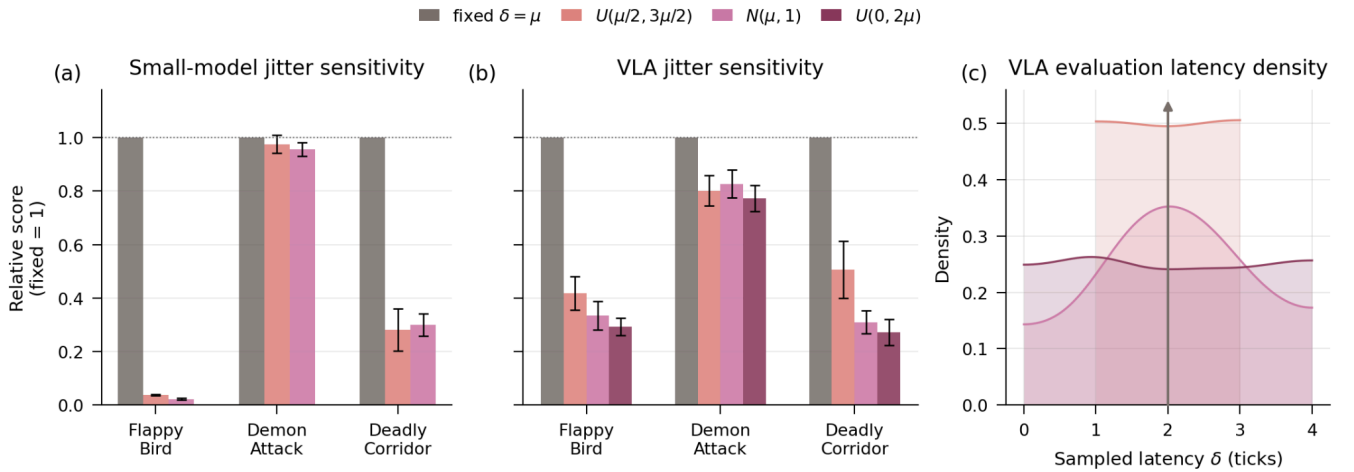


Figure 4. Fixed-latency specialization in OpenVLA. Each matrix evaluates policies trained at different fixed latency conditions across the evaluation grid. Matched or nearby conditions dominate, although the pattern and asymmetry vary by task. Source: exp-review.html.

3.4 Latency variance hurts performance even at matched mean latency

The previous experiments show that training across broader latency conditions can improve robustness. However, they leave open which properties of the deployment profile must actually be reproduced during training. In particular, is a stochastic latency profile equivalent to a fixed delay with the same mean?

To isolate the effect of latency variability, we compare fixed and stochastic evaluation profiles while holding the mean latency constant.



3.5 Profile-conditioned training and explicit latency information

The training intervention has two complementary parts. First, the policy should experience a distribution of latency conditions that covers the intended deployment profiles. Second, a single policy operating across profiles needs information about the current timing condition. Mixed-latency training improves nonzero-latency performance in the current experiments, while removing the latency prompt causes the

Flappy Bird policy to collapse. The safe conclusion is that latency diversity and explicit conditioning are both important; the ablation does not prove that text conditioning is sufficient in every task.

Figure 5. Training latency-aware policies. (a) Explicit latency information is critical under mixed-latency Flappy Bird training. (b) Same-task mixed-latency training improves delayed evaluation, with task-dependent effects at latency zero. Source: standalone.html.

3.6 Selective cross-task transfer

Cross-task data can reduce the amount of target-task training only when the companion task transfers the relevant timing structure. The current results suggest a comparatively strong relationship between Deadly Corridor and Demon Attack, while Flappy Bird is often a weak transfer recipient. The main text should report a compact recipient–companion matrix; the complete scenario curves belong in the appendix.

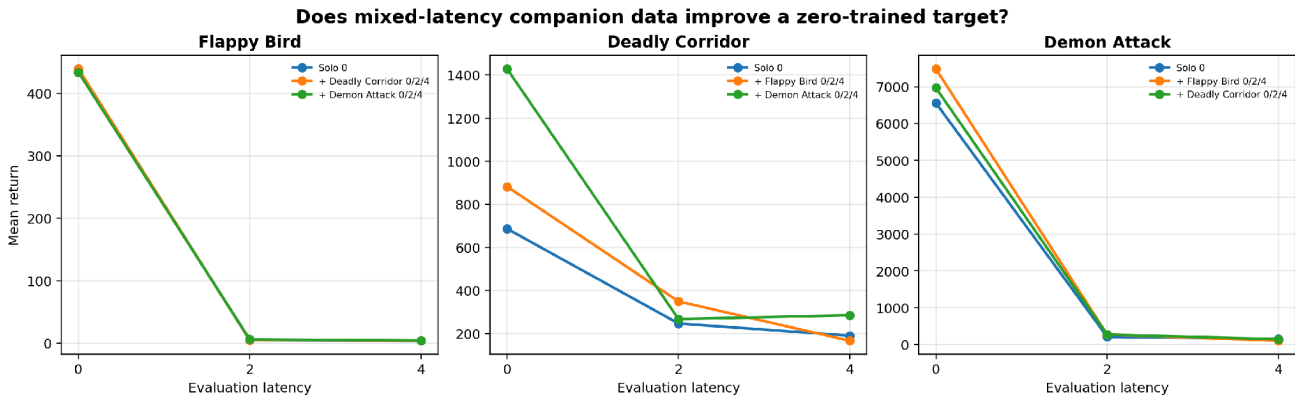


Figure 6. Representative cross-task experiment: a zero-latency target combined with a mixed-latency companion. Transfer is asymmetric and depends on the recipient–companion pair. The complete set of scenario curves should move to the appendix. Source: standalone.html.

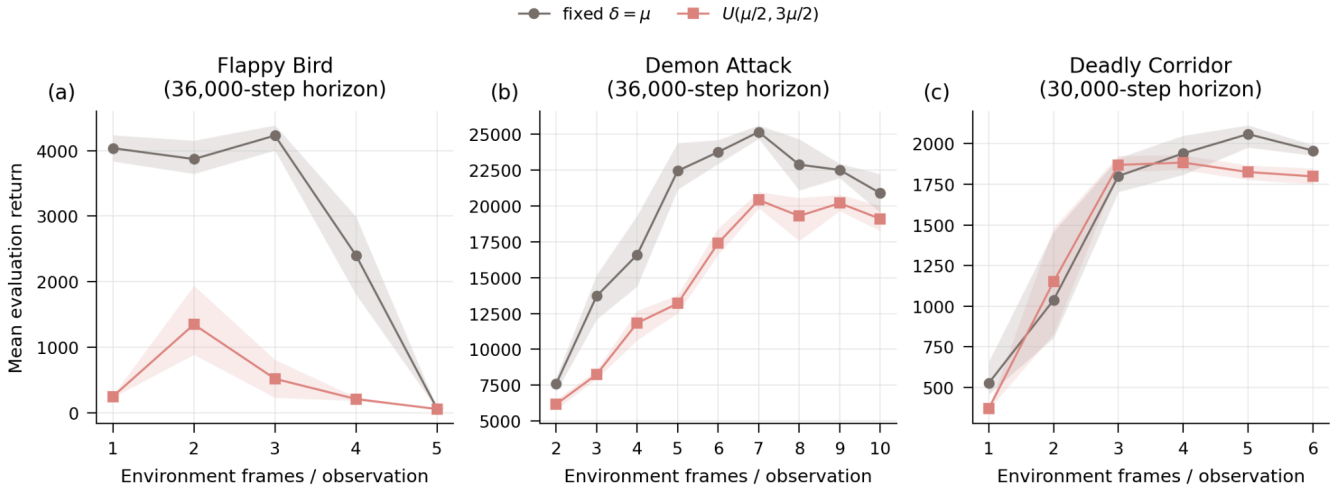
TODO: add single-task, multi-variation experiments here

4. Temporal Context and Representation for Latency-Aware Control

Profile conditioning tells an agent about the deployment delay, but it does not fully reconstruct the current physical state from a stale observation. Temporal memory can provide motion, change, and action-timing cues. This section studies whether history improves control and whether timing information is accessible inside the learned representation.

4.1 Small-model study: temporal context has a task-dependent resolution and span

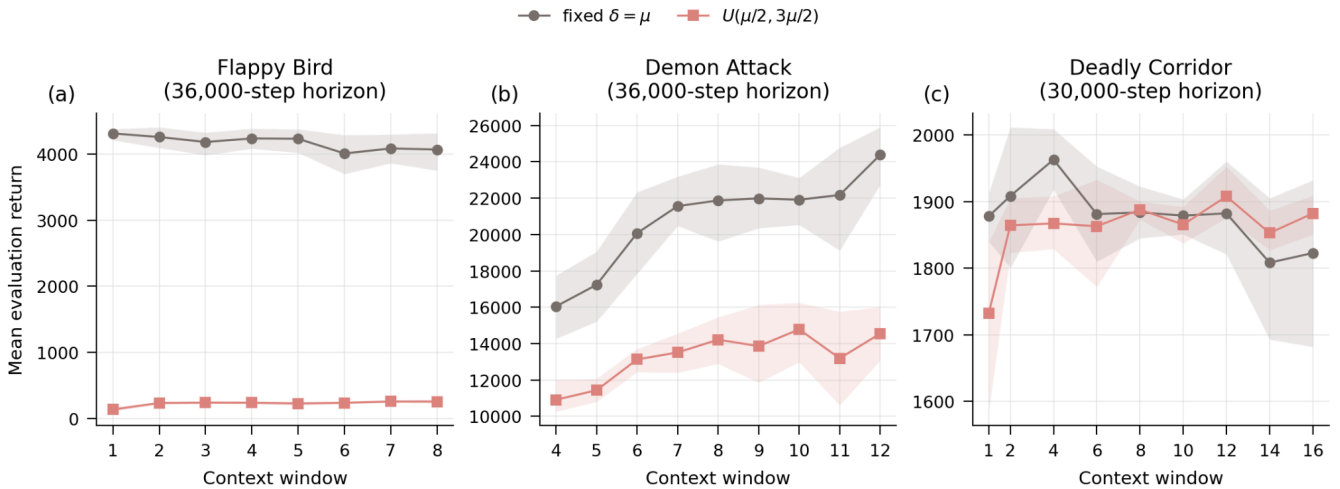
Latency makes the latest observation stale, but compensating for staleness does not necessarily require maximally frequent observations. We first vary the number of environment frames between consecutive observations. Performance changes non-monotonically with observation cadence, and an intermediate stride often performs best. Moreover, the best observed cadence differs across environments and between fixed and stochastic latency profiles.



This pattern is consistent with a trade-off between temporal resolution and temporal coverage: dense observations preserve fine-grained dynamics, whereas more widely spaced observations can expose changes over a longer physical interval.

Observation cadence determines the temporal spacing of the available evidence, but not how much evidence the policy retains. We therefore next vary the number of historical observations provided to the agent.

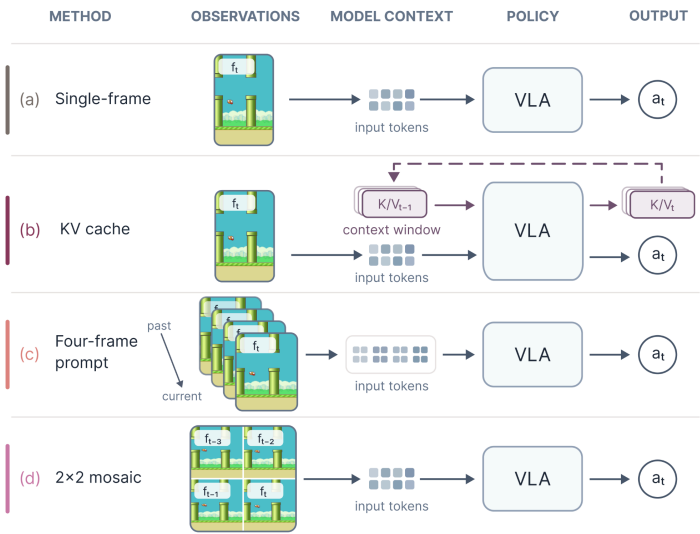
Increasing the context window produces similarly non-uniform effects. Additional history substantially improves Demon Attack over part of the sweep, while producing limited or non-monotonic gains in Flappy Bird and Deadly Corridor. Longer temporal context is therefore useful only to a limited extent, and its value depends on both the environment dynamics and the latency condition.



Together, these experiments show that temporal context should be designed along two axes: observation cadence determines the temporal resolution of the input, while context length determines its temporal extent. Neither denser observations nor longer histories are uniformly optimal.

4.2 Scaling temporal memory to VLAs

The small-model study establishes that delayed control can benefit from carefully chosen temporal context, but it does not determine how that context should be represented in a VLA. We therefore compare several practical VLA memory interfaces.



TODO: need merging

Table 1: **Memory-input comparison across environments.** Cells report checkpoint-4000 mean episode return $\pm$ episode-level Student-*t* 95% confidence halfwidth over 100 fixed-seed evaluation episodes.

Method	Flappy Bird ($\delta=3$)	Demon Attack ($\delta=6$)	Deadly Corridor ($\delta=6$)
Single frame	368.9 $\pm$ 25.9	5671.8 $\pm$ 126.3	253.1 $\pm$ 36.7
Four-frame prompt	352.1 $\pm$ 28.2	5843.7 $\pm$ 134.0	170.7 $\pm$ 31.5
2x2 mosaic	343.1 $\pm$ 29.5	6123.9 $\pm$ 100.3	255.7 $\pm$ 36.9
KV memory (Flex)	382.2 $\pm$ 25.1	6483.5 $\pm$ 109.9	175.2 $\pm$ 33.6

+zeju’s wan oft

Method	Environment	Latency	Return $\uparrow$
		raw frames / ms	mean $\pm$ 95% CI
WanOFT	Flappy Bird	3 / 100.0	425.3 $\pm$ 30.2
WanOFT	Demon Attack	6 / 100.0	4,608.8 $\pm$ 652.4
WanOFT	Deadly Corridor	6 / 171.4	238.9 $\pm$ 67.8

+talha’’s ghost trail

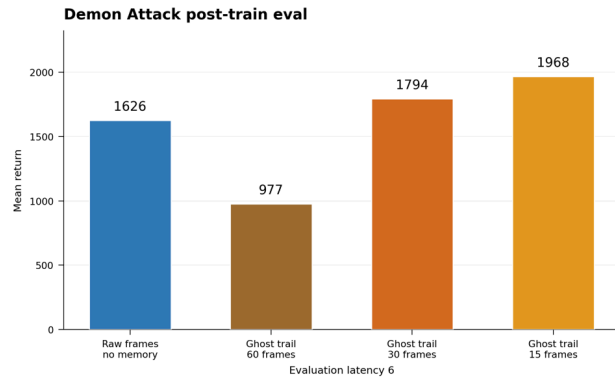
4.3 Timing is represented, but not always used robustly

Linear probes provide the key mechanistic distinction. Correctly ordered visual history makes time-to-action information accessible in intermediate representations; shuffling frames or repeating the current frame removes much of the signal. Removing the explicit latency prompt has little effect on selected visual probes, indicating that the representation derives timing primarily from ordered visual change. Later action-query

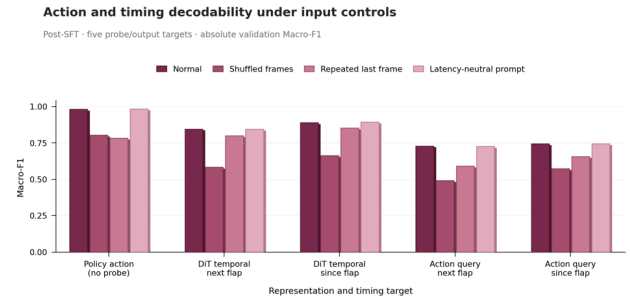
representations retain less timing information, and strong decodability does not automatically produce latency-invariant control.

+ Zeju's results on WAM models

(a) Memory/ghost trail pilot



(b) Timing decodability under input controls



(c) Timing decodability by distance bucket

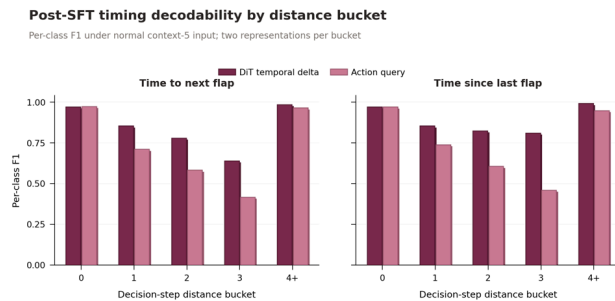
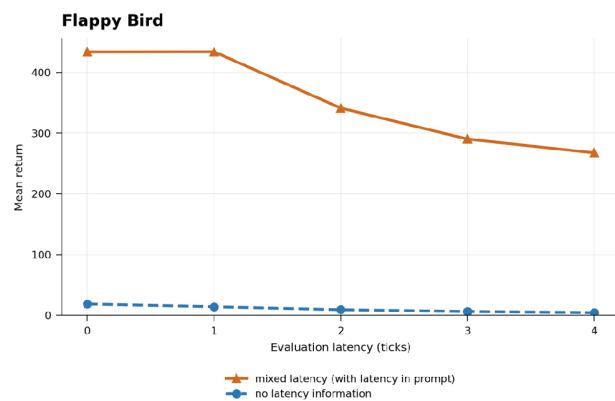


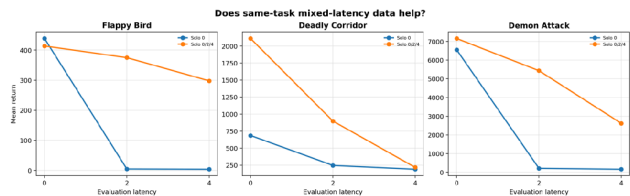
Figure 7. Memory and representation analysis. (a) A provisional memory pilot on Demon Attack. (b) Timing decodability under ordered, shuffled, repeated-frame, and no-latency-text inputs. (c) Timing decodability by decision-step distance. Sources: standalone.html and exp-review.html.

4.4 is language latency representation (in prompt) useful?

(a) Is latency conditioning



(b) Is latency-based mixed-latency training



5. Recommended Final Contribution Framing

The paper should present one integrated system and scientific story rather than a profiling paper followed by unrelated ablations. Profiling defines and predicts the deployment condition; profile-conditioned training creates the latency-aware agent; memory and representation analysis explain the remaining gap between knowing the clock and acting correctly under it.

Contribution	Paper claim	Required evidence
1. Measurement and prediction	A standardized protocol captures device-specific temporal latency profiles and replays them to predict deployment policy performance and ranking.	Primary validation: cross-device profile diversity and real-versus-replay policy results.
2. Latency-aware agent training	Target profiles become training and checkpoint-selection conditions, reducing the need for large-scale real-device data collection.	Primary validation: zero/fixed/mixed/target-profile training and explicit-conditioning ablations.
3. Memory and representation analysis	Temporal history makes timing information accessible, but information availability alone does not guarantee robust action selection.	Primary validation: memory formats, context length, input perturbations, and representation probes.

Table 2. Final contribution structure. The three claims correspond directly to the three-part paper narrative and determine which results must remain in the main text.

Recommended main-paper figure order

Number	Main message	Status
Figure 1	Latency-aware agent big picture	New overview schematic
Figure 2	Profile standardization and policy-based profile evaluation	Replace draft schematic with Xinyuan's original device/profile panels
Figure 3	Cross-model latency degradation	Existing original figure
Figure 4	Fixed-latency train–evaluation matrices	Existing original heatmaps
Figure 5	Profile-conditioned training and latency information ablation	Existing original plots
Figure 6	Compact cross-task transfer matrix or one representative scenario	Existing curves; full set in appendix
Figure 7	Memory and timing representation analysis	Existing original plots

Highest-priority missing evidence

- Export and insert Xinyuan’s original Slide 7 profiling panels at full resolution.
- Report paired real-device and replayed returns for multiple policies, including policy-ranking agreement.
- Show that profile-conditioned training selects or produces the policy that performs best on the target hardware.
- Align environment versions, action-step versus raw-frame units, frame skip, evaluation horizon, and seed reporting.
- Keep Ghost Trail and LoRA claims provisional until rerun under the final common configuration.

Appendix

Appendix A. Efficient KV-memory Training and Inference Details

This appendix describes the implementation of our streaming KV-memory policy. We first specify the token layout used during online inference and explain how cached visual keys are repositioned under a bounded sliding window. We then present a serial training procedure that faithfully reproduces streaming inference. Finally, we describe a packed implementation that replaces repeated small forward passes with a single masked forward while preserving the same per-step visibility and positional structure.

Appendix A.1 Token Layout and Cache Updates during Streaming Inference

At each decision step, the model receives four token regions: static text tokens, cached historical visual tokens, newly encoded tokens for the current frame, and an action-query token. The static instruction is placed at the beginning of the sequence and is never evicted. Historical visual frames are retained in a bounded KV buffer, while the current frame is encoded once and appended to the visible context. The action query is placed at the end of the sequence so that it can attend to the instruction and all currently visible visual frames.

After producing the action, the current visual KV is inserted into the historical buffer. Once the buffer reaches its maximum capacity, the oldest visual frame is evicted before the next frame is added. Thus, for a visual context window of size WWW , each action query observes the current frame together with at most $W-1$ cached historical frames.

inference token layout

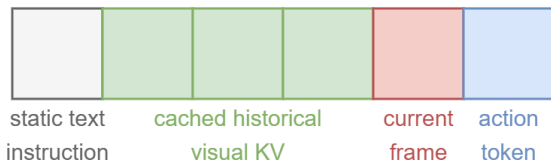


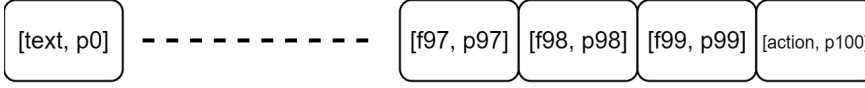
Figure A1. Streaming inference token layout. Static instruction tokens form a persistent prefix. Historical visual KVs are maintained in a bounded cache, the current frame is newly encoded at every step, and the final action-query token attends to the complete visible context.

Keeping the static text region at the beginning has two practical advantages. First, its KV states can be reused across the episode. Second, the stable prefix provides an attention-sink region that remains present as visual tokens enter and leave the sliding window.

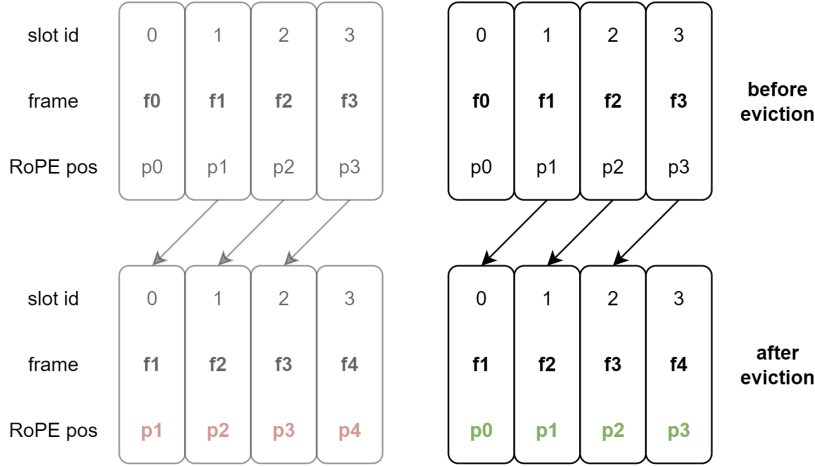
Appendix A.2 RoPE Re-rotation under Sliding-Window Eviction

A naive KV cache assigns each visual frame a position that increases with episode time. When the oldest frame is evicted, the remaining cached frames retain their original RoPE positions. Consequently, the visible window may contain positions such as $p_{97}, p_{98}, p_{99}, p_{\{97\}}, p_{\{98\}}, p_{\{99\}}$, even though these frames now occupy the first three slots of a short local context. The positional indices therefore continue to drift with episode length and no longer reflect the compact geometry of the active sliding window.

We instead store each visual key in a canonical pre-RoPE form. At every inference step, cached keys are re-rotated according to their current slots in the visible window. After an eviction, a frame that moves from the second slot to the first slot is reassigned from position p_{-1} to p_{-0} , and the newly observed frame is placed at the final slot. This keeps visual positions bounded by the context-window size and makes the positional structure independent of total episode length.



Naive sliding window vs. re-rotation



Appendix A.3 Faithful Serial Streaming Training

The most direct training procedure unrolls the policy step by step and reproduces the online cache update exactly. Each training example contains R consecutive decision steps. At step t , the model newly encodes frame f_t , combines it with the cached KVs of the preceding visible frames, predicts action a_t , and then updates the cache for the next step.

We apply supervision at every valid decision step rather than only at the final step. For a context window of size W , early steps represent the cache-filling phase, while later steps represent steady-state streaming with repeated eviction. We therefore choose a rollout length $R > W$, ensuring that training exposes the model to both regimes.

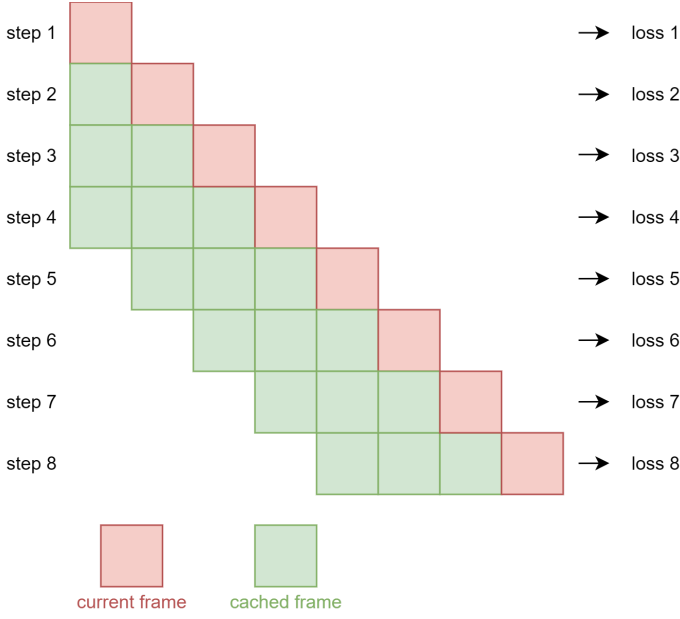


Figure A3. Serial streaming-memory training. Each rollout is processed one decision step at a time. The current frame is newly encoded, recent visual KVs are reused, and an action loss is applied at every valid step. Choosing $R > WR > WR > W$ covers both cache filling and steady-state eviction.

In our experiments, the visual context window is $W=4$, corresponding to the current frame and up to three cached historical frames, while the rollout length is $R=8$. The training objective is

$$\mathcal{L} = \frac{1}{|\mathcal{V}|} \sum_{t \in \mathcal{V}} \ell(\hat{a}_t, a_t),$$

where $\mathcal{V}$ contains valid, non-padding decision steps.

This serial implementation provides the reference semantics for our memory policy: every action is computed from exactly the information that would be available during online deployment. It is therefore useful both as a correctness baseline and as a target against which the packed implementation can be validated.

Appendix A.4 Training acceleration

Although serial unrolling faithfully matches online inference, it is inefficient on modern accelerators. A rollout of length R requires R separate forward passes, each operating on a relatively small batch and a short sequence. This produces poor GPU utilization and repeatedly incurs Python control-flow, cache assembly, kernel-launch, and per-layer re-rotation overheads. Moreover, cache-enabled execution complicates the use of standard training optimizations such as gradient checkpointing.

We eliminate the Python-level streaming loop by packing the entire rollout into one sequence. Frames $f_0, \dots, f_{R-1}$ and their corresponding action-query tokens are flattened along the sequence dimension and processed in a single forward pass. Each visual frame is encoded once, and all per-step action predictions are produced together.

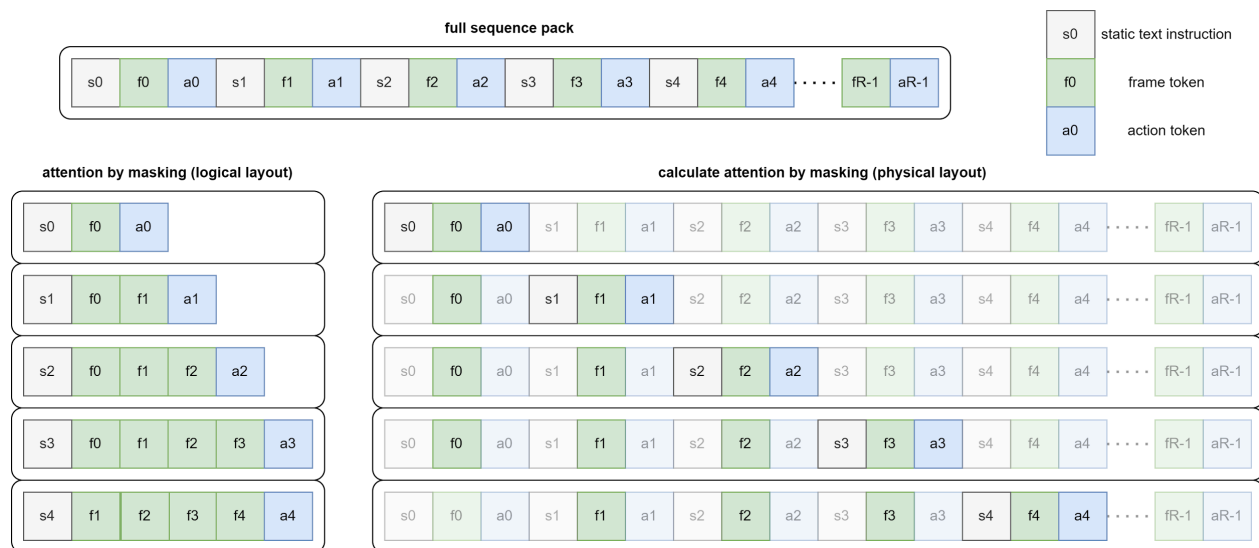
Streaming causality is enforced through an attention mask. The action query for step t may attend only to the static instruction, the visual frames contained in its active window, and its own action-query position. It cannot attend to future frames, future action queries, or frames that would already have been evicted during online inference.

Attention masking alone does not fully reproduce online inference. In a single physical packed sequence, the static text prefix occupies one fixed position and frame positions increase with rollout time. During actual streaming inference, however, every eviction compacts and re-bases the active visual window. Thus, two tokens can have the same visibility pattern but different RoPE-relative positions in packed training and online inference.

To reproduce the online positional layout, we instantiate a logical text-sink region for every rollout step. The action query at step t attends only to its own sink sts_t , its active visual window, and its own action token. The sink is assigned the position immediately preceding the oldest visible frame, while the visible frames are assigned compact local positions following the same convention as streaming inference.

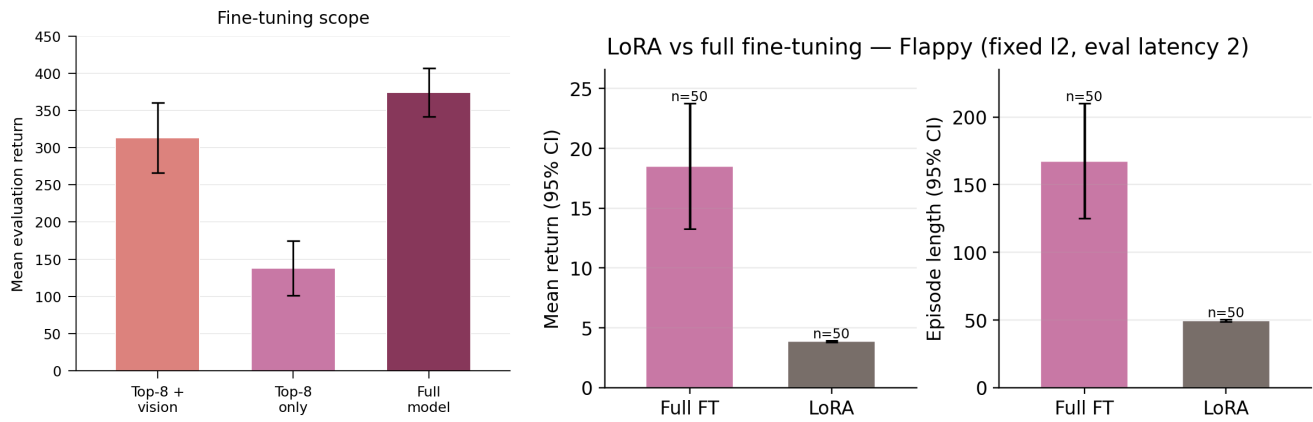
Although all sinks and frames are stored in one physical packed sequence, the attention mask and custom position IDs create a separate logical sliding-window layout for every action query. This construction emulates both eviction and positional re-basing within a single forward pass.

The resulting packed computation preserves the per-step visibility and RoPE geometry of serial streaming execution while substantially improving GPU utilization. It also avoids repeatedly assembling dynamic caches and enables the entire rollout to participate in one differentiable forward computation.



Appendix B. Fine tuning scope — why we choose full tuning instead of only top layers or LoRA?

(not sure if we need this section; if so, can re-run lora exp and add that to left figure)



TBD

Open-source effort:

Think about use cases & best code interface: Talha Zeju Xinyuan Zihan

- Paper writing: zihan
- Given new machine, very easily get the profile & train a model on another machine: Talha
- Env guide: given a new environment written in Gym, how to re-write it as latency sensitive environment: Zeju
- Live demo (videos): Xinyuan: 3 envs, 5 gpus, show how model is playing cut3r
- Readme: Talha Zeju Xinyuan